

RouteRight — Final Report

RouteRight: A Yale Student-Support Intent Classifier

Course: CPSC 381/581 Machine Learning — Final Project **Team:** Ronald Milgo, Abel Adugna, Kevin Rusagara

*Working draft. The baseline pipeline (TF-IDF + logistic regression) is fully reported below; the Comparison with NB / Linear SVM sub-section in **Results** is reserved for Abel's contribution and is currently a stub.*

1. Introduction

Yale students send thousands of short questions per term through email forms, helpdesk channels, and chatbots. Even small misroutings — a financial-aid question that lands in the registrar's inbox, a dining-services question that ends up with IT — produce delayed answers and silent rework for staff. Hand written keyword rules cover the easy cases but break on phrasing variation ("FAFSA appeal" vs "scholarship review"), multi-office wording, and informal student English.

Problem. Given a short student question, predict which of six Yale support offices should triage it: Registrar, Financial_Aid, Housing, IT, Dining, or Health_Wellness.

Approach. We frame the task as supervised multiclass text classification over **public Yale FAQ pages**. The baseline is a TF-IDF + logistic regression pipeline, which is strong on small, clean text and gives interpretable per-class word weights. Planned comparisons (Multinomial NB, Linear SVM) are designed to slot into the same evaluation harness.

Why this matters for the course. Beyond accuracy, we focus on three properties relevant to real triage: (1) class-imbalanced behavior, where the largest office can dominate predictions; (2) **top-3 accuracy** as a routing usefulness metric, because a human or downstream router only needs the right office to appear in a small shortlist; and (3) confidence calibration, because low-confidence predictions can be deferred to a human.

2. Data

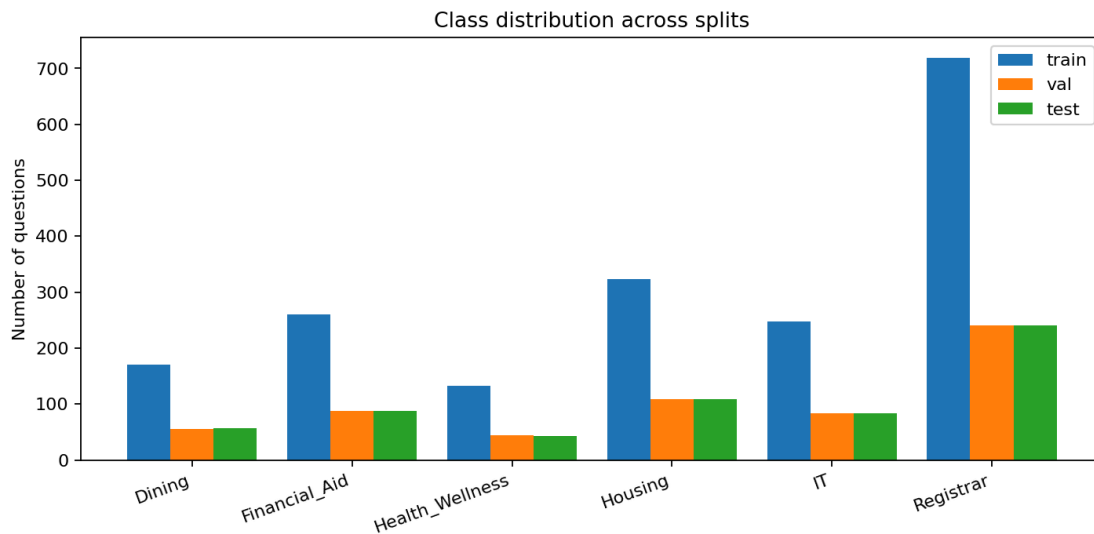
2.1 Sources

The training data is collected from **public Yale support and FAQ pages**. High-level sources are tracked in `data/raw/faq_sources.csv`; the seeds used by the polite crawler live in `data/raw/yale_faq_seed_urls.csv`. Each row in `data/raw/collected_questions.csv` carries provenance columns (`text`, `label`, `source_name`, `school`, `url`) so the report and grader can trace every example back to a Yale page.

Stage	Rows
Raw collected questions	3,089
After cleaning + dedupe (<code>src/clean_dataset.py</code>)	3,089
Train / Val / Test (60 / 20 / 20, stratified)	1,853 / 618 / 618
Schools represented	Yale only (no supplemental rows used)

Class distribution. All six offices are represented but with substantial imbalance — Registrar is roughly **5.5×** the smallest class, Health_Wellness:

Label	Train	Val	Test	Total
Registrar	719	240	240	1,199
Housing	323	108	108	539
Financial_Aid	260	87	87	434
IT	248	83	83	414
Dining	170	57	57	284
Health_Wellness	133	43	43	219



Class distribution across splits

2.2 Labeling rules

`reports/labeling_rules.md` defines six **mutually exclusive** office labels. For ambiguous multi-intent text, we assign the office that would *first* triage the case. Borderline rows were either skipped (and noted for error analysis) or labeled by the office page that the text was extracted from.

2.3 Cleaning and splits

`src/preprocess.clean_text` lowercases, trims, and collapses whitespace; we intentionally keep punctuation so TF-IDF preserves word boundaries. `src/clean_dataset.py` further enforces the six allowed labels, removes empty rows, and deduplicates by cleaned text. `src/split_data.py` produces a **stratified 60 / 20 / 20** split with `random_state=42` so re-runs are reproducible.

2.4 EDA highlights

A full exploratory analysis lives in `notebooks/exploration.ipynb`. Key numbers that inform the modeling choices:

- **Length.** Median 10 tokens, 95th percentile 48 tokens. Per-class medians cluster at 10–11 except Dining (7), so length carries almost no class-discriminating signal except to single Dining out.
- **Vocabulary.** ~4.6K unigrams and ~28K unigram+bigram features after `min_df=2`. The smallest class still has a healthy type/token ratio — the recall problem is a *prior* problem, not a vocabulary-shortage problem.

- **Cross-class overlap.** Pairwise Jaccard overlap of unigram vocabularies is ~0.20 between Registrar and most other classes, and 0.21 between Financial_Aid ↔ Health_Wellness. These are the same pairs that dominate the confusion matrix in §5.3.
- **Source diversity.** Registrar draws on 62 distinct URLs and Housing on 52, but **Health_Wellness has only 16 distinct URLs** — its training signal is narrower than the row count suggests, which compounds the class imbalance. Concrete next-iteration item: expand crawl seeds for Health_Wellness.

3. Methodology

3.1 Learning formulation

Supervised multiclass classification: input is a single short question, output is one of six office labels.

3.2 Features

We use `TfidfVectorizer` with:

- `ngram_range=(1, 2)` — unigrams and bigrams, since multiword phrases like “*net id*” and “*financial aid*” are highly office-discriminative.
- `min_df=1` — small dataset ($\approx 1,850$ train rows); we keep rare terms.
- `sublinear_tf=True` — log-scale term frequency, dampens long policy lines.

3.3 Models

- **Baseline (this report):** `LogisticRegression(solver="lbfgs", max_iter=5000)` in a scikit-learn `Pipeline` with the vectorizer above. We use the default L2 penalty and `multi_class="auto"`, which selects the multinomial loss for multiclass.
- **Planned comparisons (Abel):** Multinomial Naive Bayes and Linear SVM with `CalibratedClassifierCV` (so `predict_proba` is available for top-k metrics). These reuse the *same* train/val/test splits and are evaluated through `src.evaluate.evaluate_model`.

3.4 Metrics

We report accuracy, **macro F1** (treats every office equally — important under imbalance), weighted F1 (matches a typical user’s experience), per-class precision/recall/F1, and **top-3 accuracy** (the true office is in the model’s top three predictions). Top-3 is our headline routing-usefulness metric: a shortlist with the right office is good enough for many triage workflows.

4. Implementation Details

Component	Path	Notes
Text cleaning	<code>src/preprocess.py:clean_text</code>	lowercase, strip, whitespace
Dataset cleaning	<code>src/clean_dataset.py</code>	label whitelist + dedupe
Data crawling	<code>src/collect_yale_faqs.py</code>	polite, robots.txt, slow delay
Splits	<code>src/split_data.py</code>	stratified 60/20/20, seed=42
Baseline training	<code>src/train_baseline.py</code>	TF-IDF (1, 2), <code>min_df=1</code> , <code>sublinear_tf=True</code> + logistic regression (<code>max_iter=5000</code>)
Comparison models	<code>src/train_models.py</code>	NB / Linear SVM (Abel)
Evaluation	<code>src/evaluate.py</code>	accuracy, macro/weighted F1, per-class, top-3
Demo	<code>src/demo.py</code>	top-k CLI with confidence bars, <code>--stdin</code> , <code>--interactive</code> , <code>--model</code>
Figures	<code>src/figures.py</code>	every figure in this report
Trained model	<code>results/logistic_regression_tfidf.joblib</code>	gitignored, regenerate
Metrics dump	<code>results/metrics_baseline.json</code>	gitignored, regenerate

Reproducibility. From a fresh clone:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m src.clean_dataset
python -m src.split_data --input data/processed/final_dataset.csv
python -m src.train_baseline
python -m src.figures
```

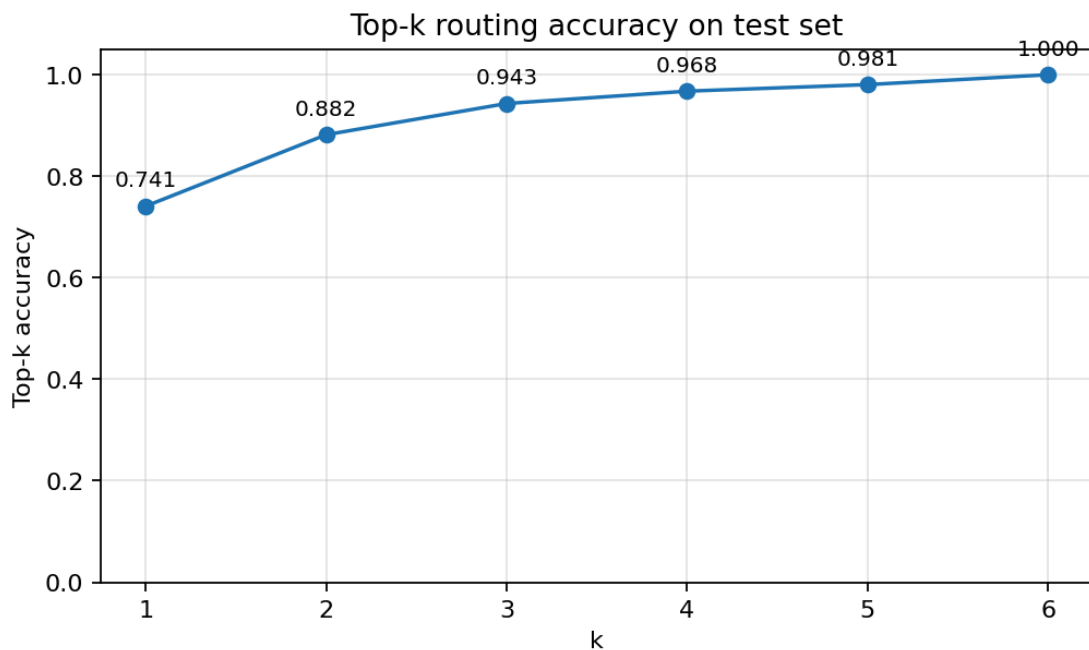
5. Results

5.1 Headline numbers (TF-IDF + Logistic Regression)

Split	Accuracy	Macro F1	Weighted F1	Top-3 accuracy
Train	0.927	0.914	0.925	0.999
Validation	0.769	0.722	0.760	0.953
Test	0.741	0.699	0.732	0.943

The 19-point train / test accuracy gap and the F1-macro vs weighted gap point at two real phenomena: (a) overfitting on a high-dimensional sparse TF-IDF space with only ~1.8K training rows, and (b) class imbalance that pushes weighted scores up relative to macro scores.

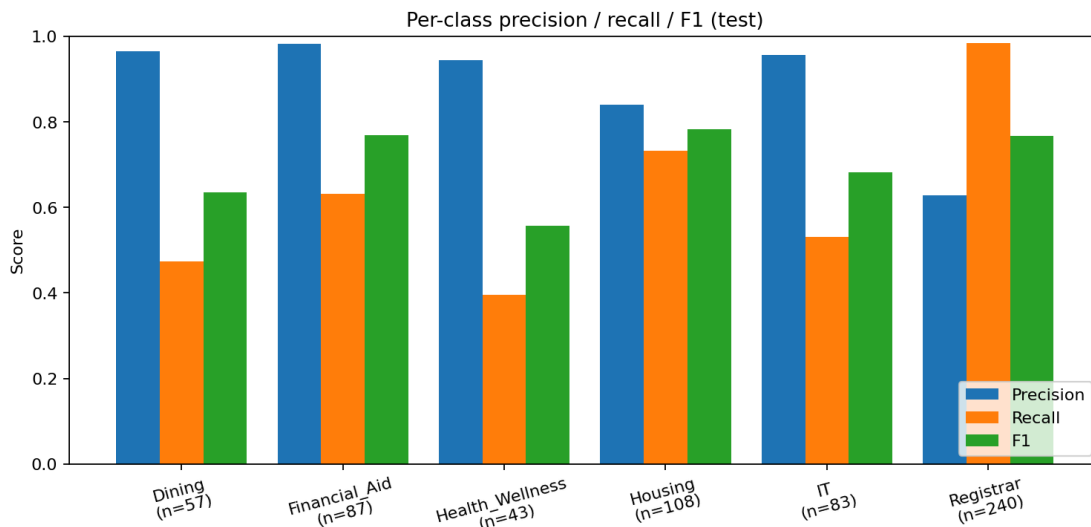
Top-k routing accuracy (test):



Top-k routing accuracy

The shape of this curve is the most actionable result for deployment: even when the top-1 prediction is wrong, the right office is in the top-3 in **94.3%** of cases and the top-5 in **98.1%**. A real triage agent would benefit far more from a shortlist than from a single hard label.

5.2 Per-class breakdown

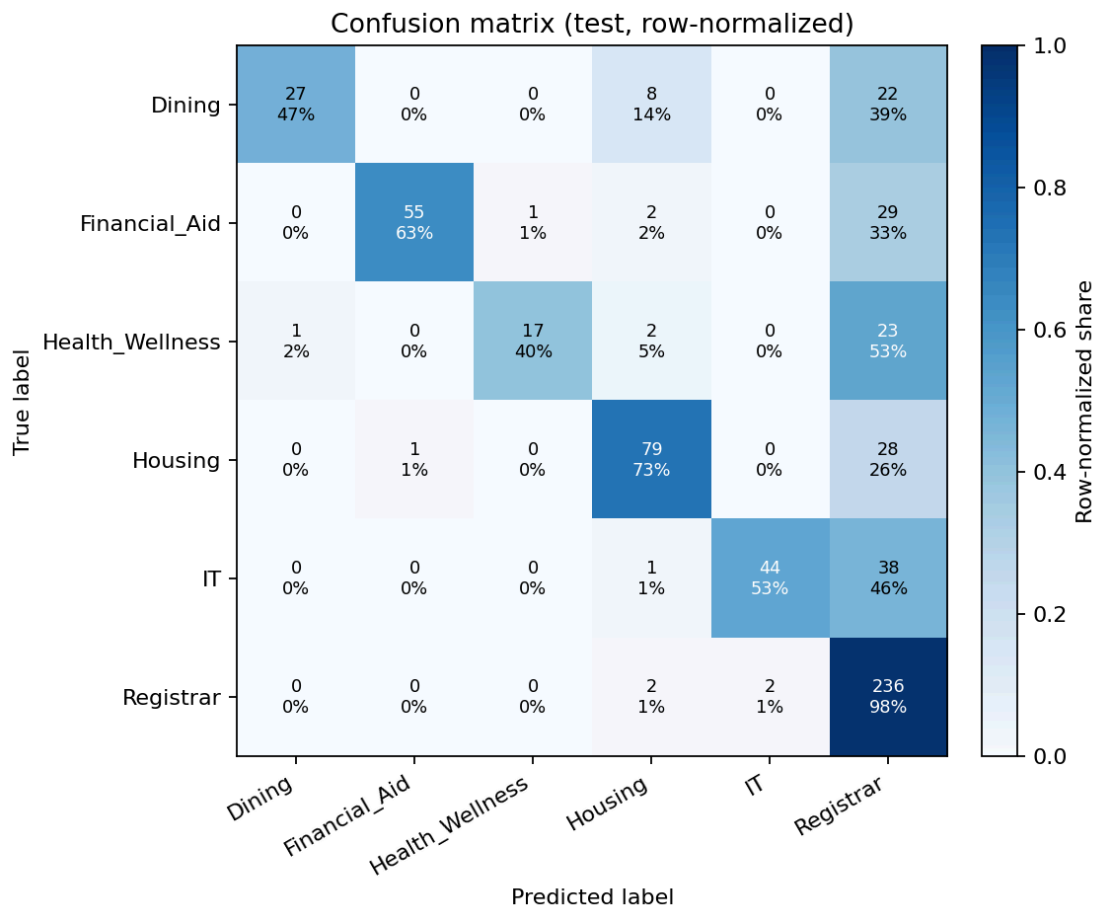


Per-class precision / recall / F1

Class	Precision	Recall	F1	Support
Registrar	0.63	0.98	0.77	240
Housing	0.84	0.73	0.78	108
Financial_Aid	0.98	0.63	0.77	87
IT	0.96	0.53	0.68	83
Dining	0.97	0.47	0.64	57
Health_Wellness	0.94	0.40	0.56	43

Every non-Registrar class is **precision-heavy and recall-light**: when the model says “IT”, it is almost always right (0.96), but it only catches 53% of true IT questions. Registrar is the inverse: it captures 98% of registrar questions but only 63% of its predictions are correct. This is a textbook majority-class bias amplified by Registrar being a semantically broad class (“course”, “registration”, “school” overlap with most other offices’ FAQ text).

5.3 Confusion matrix

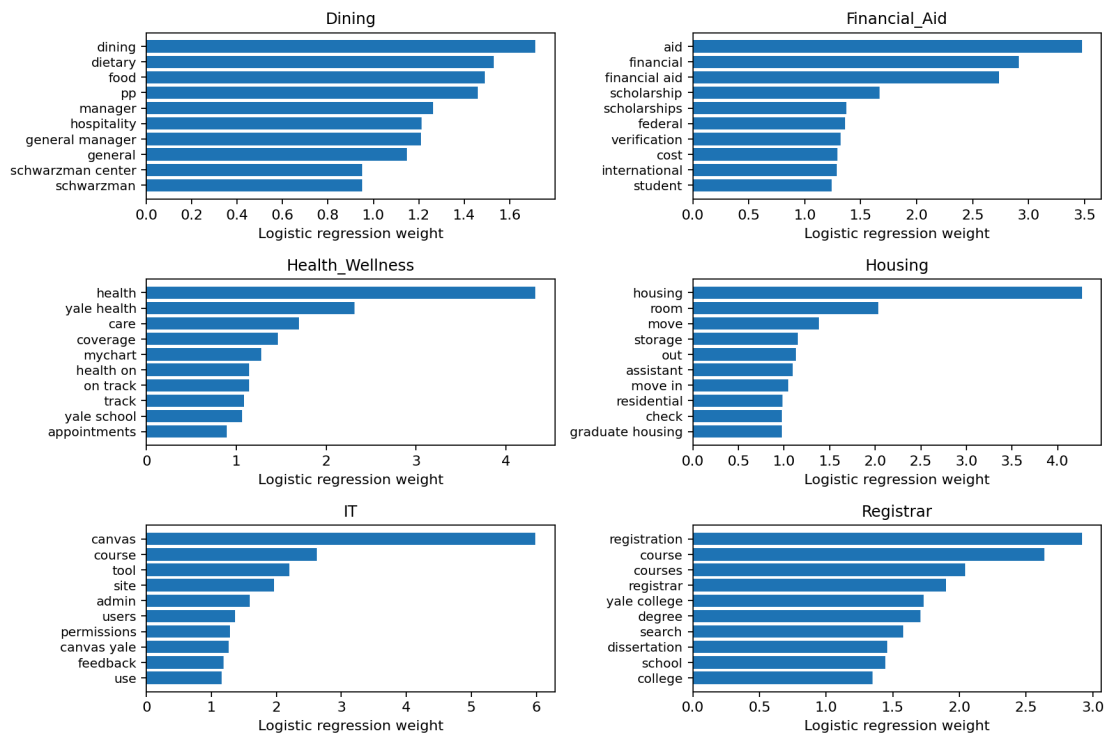


Test confusion matrix (row-normalized)

Reading the right column, between **26% and 53%** of every other class's true test examples are misrouted to Registrar. Health_Wellness loses 53% of its recall this way (23 of 43 true Health_Wellness questions get tagged Registrar). The other significant confusion is Dining → Housing (14%), which is plausible because residential dining and housing FAQs share vocabulary about meal plans tied to room type.

5.4 What the model has actually learned

Top 10 TF-IDF features per class

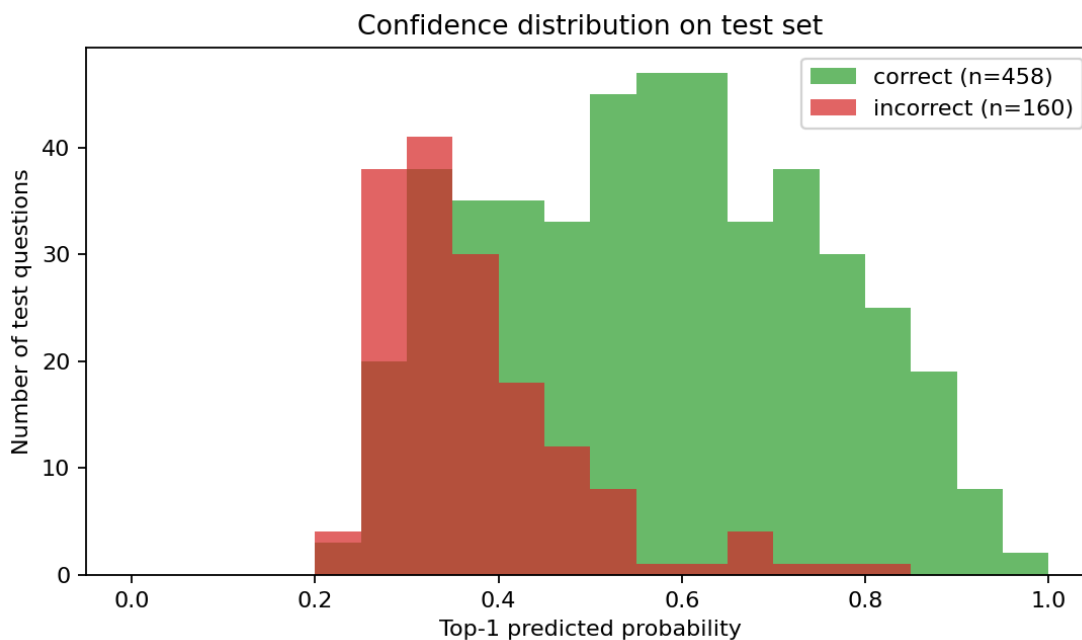


Top TF-IDF features per class

The top-weighted features are sensible, which gives confidence the model is not memorizing artifacts:

- **Dining:** *dining, dietary, food, hospitality, schwarzman center.*
- **Financial_Aid:** *aid, financial, financial aid, scholarship, federal, verification.*
- **Health_Wellness:** *health, yale health, care, coverage, mychart.*
- **Housing:** *housing, room, move, storage, residential, graduate housing.*
- **IT:** *canvas, course, tool, site, admin, permissions, canvas yale.* Note that *course* appearing strongly under IT is a known overlap with Registrar and contributes directly to the IT → Registrar confusion seen above.
- **Registrar:** *registration, course, courses, registrar, yale college, degree.*

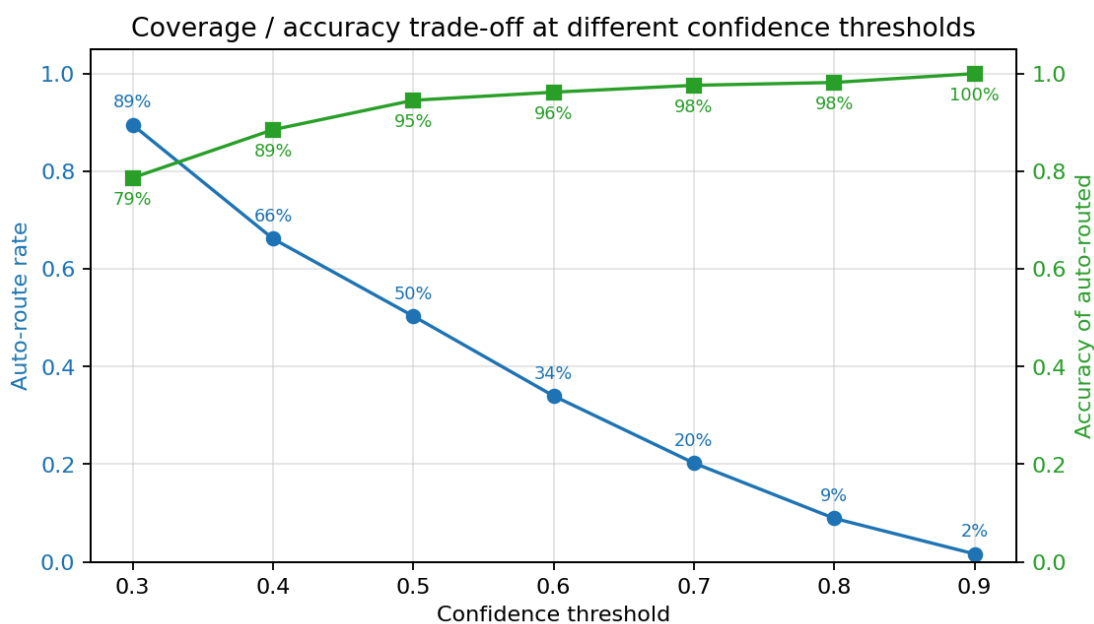
5.5 Confidence is a useful gatekeeper



Test confidence distribution: correct vs incorrect

Incorrect predictions cluster sharply at low top-1 probabilities (peak around 0.30), while correct predictions extend across the full range up to ≈ 0.95 . Practically: a confidence threshold around 0.50 would let the system auto-route high-confidence predictions and defer the rest to a human or to a top-3 shortlist UI — exactly the behavior the demo CLI exposes.

5.5.1 Coverage / accuracy trade-off (deployment view)



Coverage / accuracy trade-off at different confidence thresholds

Sweeping the threshold turns the histogram above into a deployment knob. The operating points are:

Confidence threshold	Auto-route rate	Accuracy of auto-routed	Deferred to human
0.30	89.5%	78.7%	65
0.40	66.2%	88.5%	209
0.50	50.3%	94.5%	307
0.60	34.0%	96.2%	408
0.70	20.2%	97.6%	493
0.80	8.9%	98.2%	563

A practical operating point is therefore **threshold = 0.50**: half of incoming questions auto-route at near-95% accuracy, and the remaining half can be served the top-3 shortlist for a human (or the student themselves) to pick from. That single curve replaces the abstract “use logistic-regression confidence” recommendation with a concrete service-level number a triage team could plan around.

5.6 Comparison with Naive Bayes and Linear SVM (Abel)

Pending — Abel’s `src/train_models.py` work. This sub-section will compare Multinomial NB and Linear SVM (with `CalibratedClassifierCV`) against the logistic regression baseline using the same train/val/test splits and the same `src.evaluate.evaluate_model` harness. Once `results/metrics_nb.json` and `results/metrics_linsvc.json` exist, we will:

- Append rows to the headline table in §5.1.
- Re-run `python -m src.figures --model <other>.joblib --out-dir reports/figures/<other>` to produce per-model confusion matrices and per-class bars for direct comparison.
- Discuss when each model is preferred (e.g. NB on smaller classes, SVM on top-1 sharpness).

6. Error Analysis

We complement the aggregate numbers with concrete failure modes drawn from the demo CLI and the test confusion matrix.

1. **“Reset NetID password” routes to Registrar instead of IT.** The bigram *NetID* is rare in the training set; the trigram features the model relies on for IT (*canvas yale, admin, permissions*) are dominated by the Registrar prior on words like *course* and *school*. Adding more crawl coverage of `its.yale.edu` and Yale IT support pages should fix this.
2. **“FAFSA appeal” routes to Registrar instead of Financial_Aid.** *FAFSA* itself is a high-IDF term, but in this query both *appeal* and *FAFSA* sit under a Registrar prior because much of the training *Financial_Aid* corpus uses formal phrases like “financial aid verification” rather than “FAFSA” alone. This is a wording-mismatch problem: the model has learned the *form* of FAQ language better than the *form* of student questions.
3. **Health_Wellness → Registrar (53% of true Health_Wellness).** The biggest single failure mode. Inspection of those rows shows that many Yale Health FAQ pages contain academic-calendar context (“appointments before the start of the term”, “leave of absence”) that overlaps with Registrar vocabulary. A future iteration should either (a) up-weight *Health_Wellness* under-represented classes via `class_weight="balanced"`, or (b) bring more Yale Health and counseling FAQ rows into the corpus.
4. **Dining → Housing (14% of true Dining).** Plausible domain overlap (meal plans tied to residence halls). This is less concerning because the confusion is between two operationally adjacent offices.
5. **The “Registrar trap”.** Aggregating across the whole confusion matrix, **148 of 618** test predictions land on Registrar incorrectly (24% of all test rows). Every recommendation above either rebalances the loss or broadens non-Registrar coverage in the corpus.

6.1 Representative misclassified test examples

The table below samples actual test-set errors (two per true class, drawn deterministically by `src.figures.dump_misclassified_examples`). Two patterns jump out: the model often *knows* it is uncertain (top-1 confidences cluster between 0.25 and 0.50, well inside the “defer” band of §5.5.1), and the correct label is frequently in the top-3 even when the top-1 is wrong.

True label	Predicted	Top-1 conf.	Top-3	Question
Dining	Registrar	0.31	Registrar, Dining, Housing	cocktail receptions
Dining	Registrar	0.47	Registrar, Housing, Financial_Aid	what about greek cuisine in the u.s. and is it constantly rising?
Financial_Aid	Registrar	0.30	Registrar, Financial_Aid, Housing	the last day to submit a request for review for matriculated and continuing students is march 15th of the s...
Financial_Aid	Registrar	0.44	Registrar, Financial_Aid, Housing	fafsa (yale college's code: 001426)
Health_Wellness	Housing	0.25	Housing, Registrar, Dining	bridgeport hospital – milford campus, 300 seaside avenue, milford
Health_Wellness	Registrar	0.33	Registrar, Housing, Financial_Aid	designation of patient spokesperson
Housing	Registrar	0.38	Registrar, Housing, Financial_Aid	seeking medical help/assistance if someone shows signs of alcohol poisoning.
Housing	Registrar	0.28	Registrar, Housing, Financial_Aid	current full-time opportunities
IT	Registrar	0.31	Registrar, Housing, Financial_Aid	quick start guide for instructors

True label	Predicted	Top-1 conf.	Top-3	Question
IT	Registrar	0.37	Registrar, IT, Housing	certificate or non-academic program support staff
Registrar	IT	0.26	IT, Registrar, Housing	comprehensive feedback
Registrar	IT	0.51	IT, Registrar, Housing	how long will i have access to canvas course sites if i place a course on my canvas worksheet?

Every error in the first 10 rows is a *recovered* error: the true office is already in the top-3, so a shortlist UI would have surfaced it. The last two rows are the harder case — true-Registrar examples that lost their own top-1 to IT — and they reveal that some Registrar-labeled FAQ rows are themselves crawl noise (“comprehensive feedback”) or genuinely span two offices (“canvas course sites”). Cleaning these would help precision more than any modeling change.

7. Limitations and Ethical Use

- **Yale-only corpus.** The model is trained on public Yale FAQ pages. It is not designed to generalize to other universities; supplemental rows were not used. Deploying to a different institution requires retraining.
- **No personally identifiable information.** Training data is paraphrased from public FAQs; we did not ingest tickets, student identifiers, or email content.
- **Out-of-scope deployment.** Live routing into Yale’s ticketing system is out of scope for the course. The intended artifact is a prototype CLI demo and an evaluation report, not a production endpoint.
- **Calibration.** Logistic regression probabilities are usable but not formally calibrated. A production routing system would want either Platt scaling on a held-out set or a calibrated alternative.
- **Class imbalance.** The Registrar class is ~5.5× the size of Health_Wellness; aggregate accuracy partially hides this. We report macro F1 and per-class metrics so the reader can see the imbalance directly.

8. Conclusion

A simple TF-IDF + logistic regression pipeline gets to **0.741 test accuracy** and **0.943 top-3 accuracy** on a six-way Yale support-routing task. The single biggest failure mode is over-prediction of the Registrar class, which the per-class metrics, confusion matrix, and top-feature plots all independently confirm. Practically, the right product framing is *not* “pick one office” but “produce a top-3 shortlist with a confidence threshold for auto-routing”; the CLI demo (`python -m src.demo`) already exposes that behavior. The Abel comparison section will tell us how much of the headroom is recoverable by switching the linear model rather than the data.

Appendix A — Reproducing the figures and report numbers

```
source .venv/bin/activate
python -m src.clean_dataset
python -m src.split_data --input data/processed/final_dataset.csv
python -m src.train_baseline
python -m src.figures
# Then read results/metrics_baseline.json and reports/figures/*.png
```

To rebuild `reports/final_report.pdf` from the markdown source (requires `pandoc` and either Google Chrome or another Chromium-based browser):

```
cd reports
pandoc final_report.md --standalone --css=.report_style.css \
  --embed-resources --metadata title="RouteRight - Final Report" \
  -o .final_report.html
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
  --headless=new --disable-gpu --no-pdf-header-footer \
  --print-to-pdf=final_report.pdf "file://$(pwd)/.final_report.html"
```

Appendix B — Demo CLI

```
python -m src.demo "How do I reset my NetID password?"
python -m src.demo --top-k 5 "Where do I file a FAFSA appeal?"
python -m src.demo --interactive
echo "What dining halls are open on Sunday?" | python -m src.demo --stdin
python -m src.demo --model results/some_other_model.joblib "." # for Abel's models
```